

```
// === Memory Match (js/games/memory.js) ===

/**
 * Memory Match (°_Æ•ûrL) - 5 Qs•iQsj!_
 * @author ch2077
 * @version 2.0.0
 * @date 2025-04-08 R Qe•iQsÿ 4[ù!'12[ù• Xž
 * @date 2025-04-30 OiN†grid^ \@W(desktopN f>y:N Qhv„•î~~
 *
 * ~İQx<°_Æ'M[ùÿ •ÛN$_ vøT v„rLSsm~-d
 * 5Qs-¾^!ÿ R ~$4[ù!'N-~$6[ù!'šø~$8[ù!'N [¶10[ù!'Y'^ 12[ù
 * kİQsrizË<¡eö<¡key 'M[ùQh•è•û[Æ•ÛQeN N Qs
 */
const MemoryGame = {
  allEmojis: ['ð=Ü6', 'ð=Ü1', 'ð=Ü0', 'ð>ÝŠ', 'ð=Ü;', 'ð=Ü<', 'ð=Ü(', 'ð=Ü/', 'ð=Ü.', 'ð=Ü7', 'ð=Ü8', 'ð=Ü5'],
  cards: [],
  flipped: [],
  matched: [],
  moves: 0,
  timer: null,
  seconds: 0,
  container: null,
  statsCb: null,
  level: 1, maxLevel: 5,
  levelConfigs: [
    { pairs: 4, cols: 4, name: 'R ~$' },
    { pairs: 6, cols: 6, name: 'N~$' },
    { pairs: 8, cols: 4, name: 'šø~$' },
    { pairs: 10, cols: 5, name: 'N [¶' },
    { pairs: 12, cols: 6, name: 'Y'^ ' ' },
  ],
};

init(container, statsCb) {
  this.container = container;
  this.statsCb = statsCb;
  this.moves = 0;
  this.seconds = 0;
  this.level = 1;
  this.matched = [];
  this.flipped = [];
  this.showLevelIntro(() => {
    this.startLevel();
  });
},

startLevel() {
  const cfg = this.levelConfigs[this.level - 1];
  const emojis = this.allEmojis.slice(0, cfg.pairs);
  this.moves = 0; this.seconds = 0;
  this.matched = []; this.flipped = [];
  this.cards = [...emojis, ...emojis].sort(() => Math.random() - 0.5);
  this.render(cfg.cols);
  this.startTimer();
},
```



```

showLevelIntro(cb) {
  const cfg = this.levelConfigs[this.level - 1];
  const div = document.createElement('div');
  div.className = 'level-intro';
  div.innerHTML = `<div class="level-num">{, ${this.level} Qs</div><div class="lev
el-title">${cfg.name} · ${cfg.pairs} [ü</div>`;
  this.container.style.position = 'relative';
  this.container.appendChild(div);
  setTimeout(() => { if (div.parentNode) div.remove(); cb(); }, 1500);
},

render(cols) {
  this.container.innerHTML = `<div class="memory-grid" style="grid-template-colu
mns:repeat(${cols},1fr);grid-template-rows:auto"></div>`;
  const grid = this.container.querySelector('.memory-grid');
  this.cards.forEach((emoji, i) => {
    const div = document.createElement('div');
    div.className = 'memory-card';
    div.dataset.index = i;
    div.textContent = '';
    div.addEventListener('click', () => this.flip(i, div));
    grid.appendChild(div);
  });
  this.updateStats();
},

startTimer() {
  clearInterval(this.timer);
  this.seconds = 0;
  this.timer = setInterval(() => {
    this.seconds++;
    this.updateStats();
  }, 1000);
},

updateStats() {
  if (this.statsCb) {
    const cfg = this.levelConfigs[this.level - 1];
    this.statsCb({ 'QsSa': `${this.level}/${this.maxLevel}`, 'keep': this.moves, 'M[ü
': `${this.matched.length}/${cfg.pairs}`, 'eö•ö': `${this.seconds}s` });
  }
},

flip(index, el) {
  if (this.flipped.length >= 2) return;
  if (this.matched.includes(index)) return;
  if (this.flipped.includes(index)) return;

  el.textContent = this.cards[index];
  el.classList.add('flipped');
  this.flipped.push(index);

  if (this.flipped.length === 2) {
    this.moves++;
  }
}

```



```

    this.updateStats();
    const [a, b] = this.flipped;
    if (this.cards[a] === this.cards[b]) {
      this.matched.push(a, b);
      const cards = this.container.querySelectorAll('.memory-card');
      cards[a].classList.add('matched');
      cards[b].classList.add('matched');
      this.flipped = [];
      if (this.matched.length === this.cards.length) {
        clearInterval(this.timer);
        setTimeout(() => {
          if (this.level >= this.maxLevel) { this.showWin(true); }
          else { this.levelClear(); }
        }, 400);
      }
    } else {
      setTimeout(() => {
        const cards = this.container.querySelectorAll('.memory-card');
        cards[a].classList.remove('flipped');
        cards[b].classList.remove('flipped');
        cards[a].textContent = '';
        cards[b].textContent = '';
        this.flipped = [];
      }, 600);
    }
  },
},

levelClear() {
  const div = document.createElement('div');
  div.className = 'level-intro';
  div.innerHTML = `<div class="level-num">` + this.level + `</div><div class="level-title">Q&Y •ÜQe{, ${this.level + 1} Qs</div>`;
  this.container.appendChild(div);
  setTimeout(() => {
    if (div.parentNode) div.remove();
    this.level++;
    this.startLevel();
  }, 1800);
},

showWin(full = false) {
  const div = document.createElement('div');
  div.className = 'game-over-overlay';
  if (full) {
    div.innerHTML = `<h3>ð<ß% `mUæ• Qsÿ </h3><p>• •ÇN†Qh•è ${this.maxLevel} Qsÿ u(N† ${this.moves} ke</p><button class="btn" id="replayBtn">Q•geN \@\</button>`;
  } else {
    div.innerHTML = `<h3>ð<ß% `mUæ•ÇQsÿ </h3><p>u(N† ${this.moves} key € eö ${this.seconds} yð</p><button class="btn" id="replayBtn">Q•geN \@\</button>`;
  }
  this.container.style.position = 'relative';
  this.container.appendChild(div);
  document.getElementById('replayBtn').addEventListener('click', () => {
    this.destroy();
  });
}

```



```

        this.init(this.container, this.statsCb);
    });
},

destroy() { clearInterval(this.timer); }
};

// === Whack-a-Mole (js/games/whack.js) ===

/**
 * Whack-a-Mole (bSW0Y ) - 5 Qs•iQsj!_
 * @author ch2077
 * @version 2.0.2
 * @date 2025-04-22 R Qe•iQs|û~ßÿ vîh R ep+eö•ô-PR6
 * @date 2025-05-01 O S SİW0Y i,s‡-•QsSa• XŽ
 *
 * 30y0-PeöbSW0Y ŷ 9N*m Sã-•g:Qúli
 * kİN Qsg vîh R epý •%h !'•ÇQsÿ •...eö!'~óg_
 * Qúm • ^!-•QsSaR • ŷ 1000ms!'380ms
 */
const WhackGame = {
    score: 0, misses: 0,
    moles: Array(9).fill(false),
    running: false, timer: null, spawnTimer: null,
    container: null, statsCb: null,
    bestScore: 0,
    level: 1, maxLevel: 5,
    targetScore: 50, timeLeft: 30, spawnDelay: 1000,
    levelConfigs: [
        { target: 60, time: 30, spawnDelay: 1000, name: 'piŽ«' },
        { target: 120, time: 28, spawnDelay: 800, name: 'R • ' },
        { target: 180, time: 26, spawnDelay: 650, name: 'u_rÂ' },
        { target: 250, time: 24, spawnDelay: 500, name: 'g•-P' },
        { target: 350, time: 22, spawnDelay: 380, name: 'W0rñ' },
    ],

    init(container, statsCb) {
        this.container = container;
        this.statsCb = statsCb;
        this.score = 0; this.misses = 0; this.level = 1;
        const cfg = this.levelConfigs[0];
        this.targetScore = cfg.target; this.timeLeft = cfg.time; this.spawnDelay = cfg
        .spawnDelay;
        this.moles = Array(9).fill(false);
        const saved = localStorage.getItem('whack_best');
        this.bestScore = saved ? parseInt(saved) : 0;
        this.running = true;
        this.render();
        this.showLevelIntro(() => {
            this.bindEvents();
            this.startSpawner();
            this.startTimer();
            this.updateStats();
        });
    };
};

```



```

    },

    showLevelIntro(cb) {
      this.running = false;
      const cfg = this.levelConfigs[this.level - 1];
      const div = document.createElement('div');
      div.className = 'level-intro';
      div.innerHTML = `<div class="level-num">{, ${this.level} Qs</div><div class="lev
el-title">${cfg.name} · vih ${cfg.target} R </div>`;
      this.container.style.position = 'relative';
      this.container.appendChild(div);
      setTimeout(() => {
        if (div.parentNode) div.remove();
        this.running = true;
        cb();
      }, 1500);
    },

    render() {
      let holesHTML = '';
      for (let i = 0; i < 9; i++) {
        holesHTML += `
        <div class="whack-hole" data-idx="${i}">
          <div class="whack-mole" id="mole_${i}">Ø=Ý(</div>
          <div class="whack-dirt"></div>
        </div>`;
      }
      this.container.innerHTML = `
      <div style="text-align:center">
        <div style="color:var(--text2);font-size:13px;margin-bottom:8px">p¹QÛWØÝ __R ý R+p¹
zzÝ </div>
        <div class="whack-grid">${holesHTML}</div>
        <div style="margin-top:12px;color:var(--text2);font-size:12px">g šØR : ${this
.bestScore} | }/<i• Qsep</div>
      </div>`;
    },

    bindEvents() {
      this.container.querySelectorAll('.whack-hole').forEach(hole => {
        hole.addEventListener('click', () => {
          if (!this.running) return;
          const idx = parseInt(hole.dataset.idx);
          if (this.moles[idx]) {
            this.moles[idx] = false;
            this.score += 10;
            const mole = document.getElementById(`mole_${idx}`);
            mole.classList.remove('up');
            mole.textContent = 'Ø=Ü';
            setTimeout(() => { mole.textContent = 'Ø=Ý('; }, 200);
            this.updateStats();
            if (this.score >= this.targetScore) {
              if (this.level >= this.maxLevel) { this.gameOver(true); return; }
              this.levelClear();
            }
          } else {

```



```

        this.misses++;
        this.score = Math.max(0, this.score - 2);
        this.updateStats();
    }
    });
});
},

startSpawner() {
    const spawn = () => {
        if (!this.running) return;
        const hideIdx = Math.floor(Math.random() * 9);
        if (this.moles[hideIdx]) {
            this.moles[hideIdx] = false;
            const m = document.getElementById(`mole_${hideIdx}`);
            if (m) m.classList.remove('up');
        }
        const showIdx = Math.floor(Math.random() * 9);
        if (!this.moles[showIdx]) {
            this.moles[showIdx] = true;
            const m = document.getElementById(`mole_${showIdx}`);
            if (m) m.classList.add('up');
        }
        // Chance of second mole increases with level
        if (Math.random() < 0.2 + this.level * 0.1) {
            const extra = Math.floor(Math.random() * 9);
            if (!this.moles[extra]) {
                this.moles[extra] = true;
                const m = document.getElementById(`mole_${extra}`);
                if (m) m.classList.add('up');
            }
        }
        this.spawnTimer = setTimeout(spawn, this.spawnDelay);
    };
    this.spawnTimer = setTimeout(spawn, 500);
},

levelClear() {
    clearInterval(this.timer);
    clearTimeout(this.spawnTimer);
    this.running = false;
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num"> ${this.level} Qs • Çÿ </div><div class
="level-title">Q&Y •ÛQe{, ${this.level + 1} Qs</div>`;
    this.container.appendChild(div);
    setTimeout(() => {
        if (div.parentNode) div.remove();
        this.level++;
        const cfg = this.levelConfigs[this.level - 1];
        this.targetScore = cfg.target; this.timeLeft = cfg.time; this.spawnDelay = c
fg.spawnDelay;
        this.moles = Array(9).fill(false);
        this.running = true;
        this.updateStats();
    }, 1000);
}

```



```

        this.startSpawner();
        this.startTimer();
    }, 1800);
},

startTimer() {
    this.timer = setInterval(() => {
        this.timeLeft--;
        this.updateStats();
        if (this.timeLeft <= 0) this.gameOver(false);
    }, 1000);
},

updateStats() {
    if (this.statsCb) this.statsCb({ 'QsSa': `${this.level}/${this.maxLevel}`, '_-R ':
`${this.score}/${this.targetScore}`, 'RiOY': `${this.timeLeft}s` });
},

gameOver(won = false) {
    this.running = false;
    clearInterval(this.timer);
    clearTimeout(this.spawnTimer);
    if (this.score > this.bestScore) {
        this.bestScore = this.score;
        localStorage.setItem('whack_best', this.bestScore);
    }
    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = won
        ? `<h3>ð<ß% `mUæ• Qsý </h3><p>• •ÇN†Qh•è ${this.maxLevel} Qsý g ~È_-R : ${this.score}</p><butt
on class="btn" id="replayBtn">Q•geN \@</button>`
        : `<h3>eö•ôR0ÿ </h3><p>{, ${this.level}/${this.maxLevel} Qs | _-R : ${this.score}/${
this.targetScore} | g $0: ${this.bestScore}</p><button class="btn" id="replayBtn">Q•ge
N k!</button>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
        this.destroy(); this.init(this.container, this.statsCb);
    });
},

destroy() {
    this.running = false;
    clearInterval(this.timer);
    clearTimeout(this.spawnTimer);
}
};

// === Wordle (js/games/wordle.js) ===

/**ð
 * Wordle (s SU<Ĭ)ð
 * @author ch2077ð
 * @version 1.1.0ð

```



```
* @date 2025-03-20 R rH[Æb Ð
* @date 2025-04-05 bi\U<İ^"R050N*<İÐ
*Ð
* 6k!g:0 s 5[Wkİ,ñe‡SU<İÐ
* ~ÿ,r=OM•nkcxnÿ žÄ,r=[Wkİ[XW(OFOM•n• ŷ pp,r=N W(SU<İN-Ð
* •.v0f \ e/c rit •.v0+‰æ\0†Zbß•.v0Ð
* <İ^"|‰• 50N*^8‰Á5[Wkİ<İÿ T -bSiNâ-ç-İR Ð
*/Ð
const WordleGame = {Ð
  // <İ^"ÿ • v,,•ÿf/^8‰Á<İÿ RT-dN†u Pû<İTËY ep_b_ Ð
  words: ['apple','brain','crane','dance','eagle','flame','grape','heart','image',
'joker',Ð
  'knife','lemon','magic','noble','ocean','pearl','queen','raven','stone','tiger',
',Ð
  'unity','vivid','whale','xenon','youth','zebra','bloom','cloud','dream','earth',
',Ð
  'faith','ghost','honey','ivory','jazzy','kayak','light','mango','night','olive',
',Ð
  'piano','quest','river','sugar','thumb','ultra','vapor','wheat','candy','frost',
'],Ð
  target: '', guesses: [], currentGuess: '',Ð
  maxGuesses: 6, wordLen: 5,Ð
  won: false, lost: false,Ð
  container: null, statsCb: null,Ð
Ð
  init(container, statsCb) {Ð
    this.container = container;Ð
    this.statsCb = statsCb;Ð
    this.target = this.words[Math.floor(Math.random() * this.words.length)];Ð
    this.guesses = [];Ð
    this.currentGuess = '';Ð
    this.won = false;Ð
    this.lost = false;Ð
    this.render();Ð
    this.updateStats();Ð
  },Ð
Ð
  render() {Ð
    let gridHTML = '';Ð
    for (let r = 0; r < this.maxGuesses; r++) {Ð
      gridHTML += '<div class="wordle-row" id="wrow_' + r + '">';Ð
      for (let c = 0; c < this.wordLen; c++) {Ð
        gridHTML += '<div class="wordle-cell" id="wcell_' + r + '_' + c + '"></div>';Ð
      }Ð
      gridHTML += '</div>';Ð
    }Ð
    this.container.innerHTML = `Ð
      <div style="text-align:center">Ð
        <div style="color:var(--text2);font-size:13px;margin-bottom:12px">s Qú 5 [WkİSU
<İÿ ${this.maxGuesses} k!g:0 </div>Ð
        <div class="wordle-grid">${gridHTML}</div>Ð
        <div class="wordle-keyboard" id="wordleKeys" style="margin-top:16px;max-wi
dth:360px;margin-left:auto;margin-right:auto">Ð
```



```

        ${this.buildKeyboard()})
    </div>
    <div id="wordleMsg" style="margin-top:12px;min-height:24px;font-weight:600
"></div>
    </div>`;
}

// Bind keyboard clicks
this.container.querySelectorAll('.wordle-key').forEach(btn => {
    btn.addEventListener('click', () => {
        const key = btn.dataset.key;
        if (key === 'ENTER') this.submitGuess();
        else if (key === 'DEL') this.deleteChar();
        else if (!this.won && !this.lost) this.addChar(key);
    });
});

// Physical keyboard
this._keyHandler = (e) => {
    if (this.won || this.lost) return;
    if (e.key === 'Enter') { e.preventDefault(); this.submitGuess(); }
    else if (e.key === 'Backspace') { e.preventDefault(); this.deleteChar(); }
    else if (/^[a-zA-Z]$/.test(e.key)) { e.preventDefault(); this.addChar(e.key.
toLowerCase()); }
};
document.addEventListener('keydown', this._keyHandler);
}

buildKeyboard() {
    const rows = [
        ['q','w','e','r','t','y','u','i','o','p'],
        ['a','s','d','f','g','h','j','k','l'],
        ['ENTER','z','x','c','v','b','n','m','DEL']
    ];
    return rows.map(row => `<div style="display:flex;gap:4px;justify-content:cente
r;margin-bottom:4px">` +
        row.map(k => {
            const label = k === 'DEL' ? '#+' : k === 'ENTER' ? '#î' : k.toUpperCase();
            const wide = (k === 'ENTER' || k === 'DEL') ? ' style="flex:1.5"' : ' styl
e="flex:1"';
            return `<button class="wordle-key btn small" data-key="${k}"${wide}>${labe
l}</button>`;
        }).join('') + '</div>').join('');
}

addChar(ch) {
    if (this.currentGuess.length < this.wordLen) {
        this.currentGuess += ch;
        this.updateGrid();
    }
}

deleteChar() {
    this.currentGuess = this.currentGuess.slice(0, -1);
    this.updateGrid();
}

```



```

ð
  submitGuess() {ð
    if (this.currentGuess.length !== this.wordLen) return;ð
    if (!this.words.includes(this.currentGuess)) {ð
      document.getElementById('wordleMsg').innerHTML = '<span style="color:#ff6b6b'
">SU<fN W( <f^"N-</span>';ð
      return;ð
    }ð
    this.guesses.push(this.currentGuess);ð
    const rowIdx = this.guesses.length - 1;ð
    this.revealRow(rowIdx);ð
  }ð

ð
  if (this.currentGuess === this.target) {ð
    this.won = true;ð
    document.getElementById('wordleMsg').innerHTML = '<span style="color:#00d2a0'
">ð<ß% `mUøes [üÿ </span>';ð
    document.removeEventListener('keydown', this._keyHandler);ð
  } else if (this.guesses.length >= this.maxGuesses) {ð
    this.lost = true;ð
    document.getElementById('wordleMsg').innerHTML = `<span style="color:#ff6b6b'
">{ThHf/: ${this.target.toUpperCase()}</span>`;ð
    document.removeEventListener('keydown', this._keyHandler);ð
  }ð
  this.currentGuess = '';ð
  this.updateStats();ð
},ð
ð
  revealRow(rowIdx) {ð
    const guess = this.guesses[rowIdx];ð
    const targetArr = this.target.split('');ð
    const result = Array(this.wordLen).fill('absent');ð
  }ð

ð
  // First pass: correct positionsð
  for (let i = 0; i < this.wordLen; i++) {ð
    if (guess[i] === targetArr[i]) {ð
      result[i] = 'correct';ð
      targetArr[i] = null;ð
    }ð
  }ð
  // Second pass: presentð
  for (let i = 0; i < this.wordLen; i++) {ð
    if (result[i] === 'correct') continue;ð
    const idx = targetArr.indexOf(guess[i]);ð
    if (idx !== -1) {ð
      result[i] = 'present';ð
      targetArr[idx] = null;ð
    }ð
  }ð
}ð

ð
const colors = { correct: '#00d2a0', present: '#ffa726', absent: '#444466' };ð
for (let i = 0; i < this.wordLen; i++) {ð
  const cell = document.getElementById(`wcell_${rowIdx}_${i}`);ð
  setTimeout(() => {ð
    cell.textContent = guess[i].toUpperCase();ð
    cell.style.background = colors[result[i]];ð
  }, 0);ð
}ð

```



```

        cell.style.color = '#fff';
        cell.style.transform = 'rotateX(360deg)';
    }, i * 150);
}

// Color the keyboard
const keyBtn = this.container.querySelector(`.wordle-key[data-key="${guess[i]}"]`);
if (keyBtn) {
    const current = keyBtn.style.background;
    if (result[i] === 'correct' || current === '') {
        keyBtn.style.background = colors[result[i]];
    } else if (result[i] === 'present' && current !== colors.correct) {
        keyBtn.style.background = colors.present;
    } else if (result[i] === 'absent' && current === '') {
        keyBtn.style.background = colors.absent;
    }
}
}
},
},
}

updateGrid() {
    const rowIdx = this.guesses.length;
    for (let i = 0; i < this.wordLen; i++) {
        const cell = document.getElementById(`wcell_${rowIdx}_${i}`);
        if (cell) cell.textContent = this.currentGuess[i] ? this.currentGuess[i].toUpperCase() : '';
    }
},
},
}

updateStats() {
    if (this.statsCb) this.statsCb({
        'won': this.guesses.length === this.maxGuesses,
        'lost': this.guesses.length > this.maxGuesses
    });
},
}

destroy() {
    document.removeEventListener('keydown', this._keyHandler);
}
};

```

```
// === Delta Randomizer (js/games/delta.js) ===
```

```
/**
```

```

 * N %0m2^LR~-•g:‘M^AVh (Delta Force Randomizer)
 * @author ch2077
 * @version 2.3.0
 * @date 2025-04-02 Qh-bf0e°w [žn8b riTÁepcn
 * @date 2025-04-15 R QeR`u;eHgøÿ shake+popÿ
 *
 * -•g:u b N %0m2^LR`QhYW‘M^Äÿ W0Vp0 ^rTX0 g^h°0 bœu20 Y4v0
 * epcngen•ÿ [~e¹wiki+s@[¶y>S:ÿ mµvÖN†~Ý[Æ/g:[ÆT ~SR+^AY
 * kÎN*iy0MrizĚnŮR~-•g:ÿ e/c SURi‘İe°rollgDN*iy0M
 */

```

```
const DeltaGame = {
  slots: {
```



```
map: { name:'W0Vp', icon:'Ø=Ýúþ ', items:[
  '-ðS÷Y'W] · g:[Æ','-ðS÷Y'W] · l8Y ', '••_ nª£7 · g:[Æ',',*Y)WúW0 · g:[Æ',
  ',*Y)WúW0 · ~Ý[Æ',']ôQKNÀ · g:[Æ',']ôQKNÀ · ~Ý[Æ','onlPvÑrñ · ~Ý[Æ',
  'tæ\ Wú'İ · ~Ý[Æ','e SG · e;-2','pìS: · e;-2','•/z• · keb ',
  'X XÖb · e;-2','e-•h · e;-2','~İf´w< · e;-2','e-\B · e;-2'
]},
oper: { name:'^rTX', icon:'Ø<ß-p ', items:[
  '~ϕrÿ·Qİ','Z Ý™·s<[‡f ', 'eàT ·„™• \ ', 'u¼~İ·bÜ\ e⁻','•[O _·',
  '-2Z ·'Ñ','š‡r*·z\fs-İ','"ö•ü·TÈ'İhİ','VbTÍ·WÃ_ R)','
  '‡ S;•W0 ', '†Ê·OPZ ', '‡v·f _·h... \ ',
  'rg•ŠN°•*„(','NLø•æ••9f±\ ', 'mñ„Ÿ·_|_~•WY+'
]},
gun: { name:'gªh°', icon:'Ø=Ý+', items:[
  'M4A1 z•Qûkegª','K416 z•Qûkegª','AKM z•Qûkegª','SCAR-H b e-kegª',
  'AS Val z•Qûkegª','QBZ95-1 z•Qûkegª','AK-12 z•Qûkegª','AUG z•Qûkegª',
  'Vector Q²• gª','P90 Q²• gª','MP5 Q²• gª','MP7 Q²• gª','SR-3M Q²• gª',
  'AWM rÛQûkegª','M700 rÛQûkegª','R93 rÛQûkegª','M82 SÍvhgPrÛQû',
  'M249 •{g:gª','PKM •{g:gª','S12K -0_9gª','SR-25 \ bKkegª'
]},
bp: { name:'€İS ', icon:'Ø<ß'', items:[
  '•{W<€İS ', 'N-W<€İS ', 'Y'W<€İS ', 'b g/€İS ', 'z•Qû€İS ', 'İb €İS ', 'LQ<€İS '
]},
chest: { name:'€0c ', icon:'Ø>Ý°', items:[
  '•{W<€0c ', 'N-W<€0c ', 'İW<€0c ', 'b g/€0c ', 'Ě• €0c ', '• u(€0c ', 'ryyÍ€0c '
]},
armor: { name:'bªu2', icon:'Ø=Þáp ', items:[
  '•{W<-2_9^c Lv.1','• u(b g/€İ_Ã Lv.2','HTb g/€İ_Ã Lv.2',
  'R6_ -2_9€İ_Ã Lv.3','TG-H-2_9^c Lv.3','\ bKb g/€İ_Ã Lv.3',
  'kfXë-2_9€İ_Ã Lv.4','DT-AVS-2_9^c Lv.4','MK-2b g/€İ_Ã Lv.4',
  '|¼,ñ-2_9€İ_Ã Lv.5','İW<-2_9€İ_Ã Lv.5','Hvk-2-2_9^c Lv.5',
  'HA-2'İW<-2_9^c Lv.6','ry'İQKMAS2.0 Lv.6','İNR -2_9^c Lv.6','lðwf-2_9^Au2 Lv.6'
]},
helmet:{ name:'Y4v0', icon:'&Ñþ ', items:[
  'b7Y hòt ^= Lv.1','€ _ "ϕv0 Lv.1',
  'Y SädiBXY4v0 Lv.2','MC-2_9Y4v0 Lv.2','DR0b g/Y4v0 Lv.2',
  '-2f'Y4v0 Lv.3','DAS-2_9Y4v0 Lv.3','MC201b g/Y4v0 Lv.3',
  'D6b g/Y4v0 Lv.4','DICH<-~ÃY4v0 Lv.4','GT1b g/Y4v0 Lv.4',
  'GN'İW<Y4v0 Lv.5','DICH-1b g/Y4v0 Lv.5','Mask-1"ÁXÁY4v0 Lv.5',
  'H70|¼,ñY4v0 Lv.6','GT5c c%[~Y4v0 Lv.6','DICH-9'İW<Y4v0 Lv.6'
]}
},
keys: ['map','oper','gun','bp','chest','armor','helmet'],
spinning: false,
results: {},
intervals: {},
container: null,
statsCb: null,

photoPalette: [
  'linear-gradient(135deg, #1a3a2a 0%, #2d6a3f 50%, #40916c 100%)',
  'linear-gradient(135deg, #1b2838 0%, #2a4560 50%, #3a6b8c 100%)',
  'linear-gradient(135deg, #2d1b38 0%, #5a3d6b 50%, #7b52a3 100%)',
  'linear-gradient(135deg, #3d2e1a 0%, #6b4d2d 50%, #9a6d3d 100%)',
  'linear-gradient(135deg, #1a2838 0%, #2d4a6b 50%, #3d6b9a 100%)',
  'linear-gradient(135deg, #381a1a 0%, #6b2d2d 50%, #9a3d3d 100%)',
```



```

    'linear-gradient(135deg, #1a3830 0%, #2d6b5a 50%, #3d9a80 100%)',
    'linear-gradient(135deg, #2d2d1a 0%, #5a6b2d 50%, #8a9a3d 100%)',
    'linear-gradient(135deg, #381a30 0%, #6b2d5a 50%, #9a3d7b 100%)',
    'linear-gradient(135deg, #0d2137 0%, #1a3d5c 50%, #2a6090 100%)',
    'linear-gradient(135deg, #3a1a1a 0%, #6b4040 50%, #9a5a5a 100%)',
    'linear-gradient(135deg, #1a3a1a 0%, #3d6b3d 50%, #5a9a5a 100%)',
  ],

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.spinning = false;
    this.results = {};
    this.intervals = {};
    this.render();
  },

  photoFor(slotKey, itemName) {
    const items = this.slots[slotKey].items;
    const idx = items.indexOf(itemName);
    return this.photoPalette[idx % this.photoPalette.length];
  },

  render() {
    const slotCards = this.keys.map(k => {
      const s = this.slots[k];
      return `<div class="delta-slot card" id="slot_${k}">
        <div class="delta-photo" id="photo_${k}">
          <span class="delta-photo-icon">${s.icon}</span>
        </div>
        <div class="delta-label-sm">${s.name}</div>
        <div class="delta-display" id="display_${k}">?</div>
      </div>`;
    }).join('');

    this.container.innerHTML = `
      <div style="text-align:center;width:100%">
        <div class="delta-grid" id="slotGrid">
          ${slotCards}
        </div>
        <button class="btn pulse" id="rollBtn" style="margin-top:20px;font-size:18
px;padding:14px 40px">
          ⚀ _ YĖb%YV
        </button>
        <div id="deltaResult" style="margin-top:16px;min-height:40px;font-size:15p
x;font-weight:600;color:var(--accent2)"></div>
      </div>
    `;

    document.getElementById('rollBtn').addEventListener('click', () => this.startR
oll());
  },

  startRoll() {
    if (this.spinning) return;

```



```

    this.spinning = true;
    this.results = {};
    const btn = document.getElementById('rollBtn');
    btn.disabled = true;
    btn.textContent = '0<β° b½YVN-...';
    btn.classList.remove('pulse');
    document.getElementById('deltaResult').textContent = '';

    this.keys.forEach(k => {
      const display = document.getElementById(`display_${k}`);
      const photo = document.getElementById(`photo_${k}`);
      display.classList.add('rolling');
      let tick = 0;
      this.intervals[k] = setInterval(() => {
        const items = this.slots[k].items;
        const idx = Math.floor(Math.random() * items.length);
        const item = items[idx];
        display.textContent = item;
        display.style.color = ['#fff6b6b', '#ffa726', '#ffd93d', '#00d2a0', '#54a0ff', '
#a29bfe', '#e0e0f0'][Math.floor(Math.random()*7)];
        if (photo) photo.style.background = this.photoFor(k, item);
        tick++;
      }, 60);
    });

    setTimeout(() => {
      this.keys.forEach(k => {
        clearInterval(this.intervals[k]);
        const items = this.slots[k].items;
        const pick = items[Math.floor(Math.random() * items.length)];
        this.results[k] = pick;
        const display = document.getElementById(`display_${k}`);
        display.textContent = pick;
        display.style.color = 'var(--accent2)';
        display.classList.remove('rolling');
        display.classList.add('done');
        const photo = document.getElementById(`photo_${k}`);
        if (photo) {
          photo.style.background = this.photoFor(k, pick);
          photo.classList.add('photo-done');
        }
      });

      this.spinning = false;
      btn.disabled = false;
      btn.textContent = '0<β° Q•geN k!';
      btn.classList.add('pulse');
      this.showResult();
    }, 3000);
  },

  showResult() {
    const lines = this.keys.map(k => {
      const s = this.slots[k];
      return `<div class="delta-result-item">

```



```

        <span class="delta-result-icon">${s.icon}</span>
        <span class="delta-result-name">${s.name}</span>
        <span class="delta-result-val">${this.results[k]}</span>
    </div>`;
    });
    document.getElementById('deltaResult').innerHTML = `
    <div class="delta-result-card">
        <div style="font-weight:700;margin-bottom:8px;color:var(--text)">ð=ÛË  b½YV-Ógæ</
div>
        ${lines.join('')}
    </div>
    `;
},

destroy() {
    this.keys.forEach(k => clearInterval(this.intervals[k]));
}
};

// === Sudoku (js/games/sudoku.js) ===

/**
 * Sudoku (epri)
 * @author ch2077
 * @version 1.2.0
 * @date 2025-02-10 e9u(Vpn`lõu b [Æetepri+c zz
 * @date 2025-03-05 0iN†-¾^!• béeögenerateSak{v,,•i~~
 *
 * 9x9h QÆepriÿ 3yÍ-¾^!ÿ easy/medium/hardÿ
 * u b {-lõÿ Vpn`lõ• _Ru b [Æet%ãÿ Q•c -¾^!c c%,â^rh<
 * ŠÆ<Áÿ ^L/R /[«T 1-9N 'íY
 */
const SudokuGame = {
    board: [],
    solution: [],
    given: [],
    selected: null,
    difficulty: 'easy',
    container: null,
    statsCb: null,
    errors: 0,

    init(container, statsCb) {
        this.container = container;
        this.statsCb = statsCb;
        this.selected = null;
        this.errors = 0;
        this.generate();
        this.render();
        this.updateStats();
    },

    generate() {
        const { puzzle, solution, given } = this.makePuzzle();
```



```

    this.board = puzzle.map(r => [...r]);
    this.given = given.map(r => [...r]);
    this.solution = solution;
  },

  makePuzzle() {
    // Generate solved board
    const board = Array.from({ length: 9 }, () => Array(9).fill(0));
    const fill = (b) => {
      for (let r = 0; r < 9; r++)
        for (let c = 0; c < 9; c++) {
          if (b[r][c] === 0) {
            const nums = [1,2,3,4,5,6,7,8,9].sort(() => Math.random() - 0.5);
            for (const n of nums) {
              if (this.isValid(b, r, c, n)) {
                b[r][c] = n;
                if (fill(b)) return true;
                b[r][c] = 0;
              }
            }
            return false;
          }
        }
      return true;
    };
    fill(board);

    const solution = board.map(r => [...r]);
    const puzzle = board.map(r => [...r]);
    const given = Array.from({ length: 9 }, () => Array(9).fill(true));

    // Remove cells based on difficulty
    const remove = { easy: 35, medium: 45, hard: 55 }[this.difficulty];
    let removed = 0;
    while (removed < remove) {
      const r = Math.floor(Math.random() * 9);
      const c = Math.floor(Math.random() * 9);
      if (puzzle[r][c] !== 0) {
        puzzle[r][c] = 0;
        given[r][c] = false;
        removed++;
      }
    }
    return { puzzle, solution, given };
  },

  isValid(board, r, c, n) {
    for (let i = 0; i < 9; i++)
      if (board[r][i] === n || board[i][c] === n) return false;
    const br = Math.floor(r / 3) * 3, bc = Math.floor(c / 3) * 3;
    for (let i = 0; i < 3; i++)
      for (let j = 0; j < 3; j++)
        if (board[br + i][bc + j] === n) return false;
    return true;
  },

```



```

render() {
  this.container.innerHTML = `
    <div style="text-align:center">
      <div class="sudoku-board" style="display:grid;grid-template-columns:repeat
(9,1fr);gap:0;border:3px solid var(--accent);border-radius:8px;overflow:hidden;wid
th:100%;max-width:360px;margin:0 auto">
        </div>
        <div class="sudoku-pad" style="display:grid;grid-template-columns:repeat(5
,1fr);gap:6px;max-width:300px;margin:16px auto 0">
          ${[1,2,3,4,5,6,7,8,9].map(n => `<button class="btn small num-btn" data-n
="${n}" style="font-size:18px;padding:10px">${n}</button>`).join('')}
          <button class="btn small" id="eraseBtn" style="font-size:14px;background
:var(--surface2)">dæ-d</button>
        </div>
        <div style="margin-top:12px;display:flex;gap:8px;justify-content:center">
          <button class="btn small" id="hintBtn" style="background:var(--surface2)
">cðy:</button>
          <select id="diffSelect" style="background:var(--surface2);color:var(--te
xt);border:1px solid var(--border);border-radius:6px;padding:4px 8px;font-size:12p
x">
            <option value="easy">{€SU</option>
            <option value="medium">N-{I</option>
            <option value="hard">Vð-¼</option>
          </select>
        </div>
      </div>`;

  const boardEl = this.container.querySelector('.sudoku-board');
  for (let r = 0; r < 9; r++)
    for (let c = 0; c < 9; c++) {
      const cell = document.createElement('div');
      cell.dataset.r = r;
      cell.dataset.c = c;
      const v = this.board[r][c];
      const isGiven = v !== 0;
      cell.textContent = v || '';
      cell.style.cssText = `
        aspect-ratio:1;display:flex;align-items:center;justify-content:center;
        font-size:${isGiven?'17px':'15px'};font-weight:${isGiven?'700':'400'};
        background:${isGiven?'var(--surface2)':'var(--bg)'};
        border:0.5px solid var(--border);cursor:pointer;
        color:${isGiven?'var(--accent2)':'var(--text)'};
      `;
      cell.addEventListener('click', () => this.selectCell(r, c, cell));
      boardEl.appendChild(cell);
    }

  // Number pad
  this.container.querySelectorAll('.num-btn').forEach(btn => {
    btn.addEventListener('click', () => {
      if (this.selected) {
        this.fillNumber(parseInt(btn.dataset.n));
      }
    });
  });

```



```

    });
    document.getElementById('eraseBtn').addEventListener('click', () => {
        if (this.selected) this.fillNumber(0);
    });
    document.getElementById('hintBtn').addEventListener('click', () => this.giveHint());
    document.getElementById('diffSelect').value = this.difficulty;
    document.getElementById('diffSelect').addEventListener('change', (e) => {
        this.difficulty = e.target.value;
        this.init(this.container, this.statsCb);
    });
},

selectCell(r, c, cell) {
    this.selected = { r, c, cell };
    this.container.querySelectorAll('.sudoku-board div').forEach(d => d.style.outline = 'none');
    cell.style.outline = '2px solid var(--accent)';
},

fillNumber(n) {
    const { r, c } = this.selected;
    if (this.given[r][c]) return;
    if (n === 0) {
        this.selected.cell.textContent = '';
        this.board[r][c] = 0;
    } else {
        this.selected.cell.textContent = n;
        this.board[r][c] = n;
        if (n !== this.solution[r][c]) {
            this.selected.cell.style.color = '#ff6b6b';
            this.errors++;
        } else {
            this.selected.cell.style.color = '#00d2a0';
            this.refreshAllCells();
        }
    }
    this.updateStats();
    this.checkComplete();
},

refreshAllCells() {
    const cells = this.container.querySelectorAll('.sudoku-board div');
    cells.forEach(cell => {
        const r = parseInt(cell.dataset.r), c = parseInt(cell.dataset.c);
        if (this.board[r][c] !== 0 && this.board[r][c] === this.solution[r][c]) {
            cell.style.color = '#00d2a0';
        }
    });
},

giveHint() {
    // Find an empty or wrong cell
    const empty = [];
    for (let r = 0; r < 9; r++)

```



```

        for (let c = 0; c < 9; c++)
            if (this.board[r][c] !== this.solution[r][c]) empty.push({ r, c });
        if (empty.length === 0) return;
        const { r, c } = empty[Math.floor(Math.random() * empty.length)];
        this.board[r][c] = this.solution[r][c];
        const cells = this.container.querySelectorAll('.sudoku-board div');
        cells[r * 9 + c].textContent = this.solution[r][c];
        cells[r * 9 + c].style.color = 'var(--accent2)';
        cells[r * 9 + c].style.fontWeight = '700';
        this.updateStats();
        this.checkComplete();
    },

    checkComplete() {
        for (let r = 0; r < 9; r++)
            for (let c = 0; c < 9; c++)
                if (this.board[r][c] !== this.solution[r][c]) return;
        this.showWin();
    },

    updateStats() {
        if (this.statsCb) {
            const filled = this.board.flat().filter(v => v !== 0).length;
            this.statsCb({ 'jðXk': `${filled}/81`, '• <i': this.errors });
        }
    },

    showWin() {
        const div = document.createElement('div');
        div.className = 'game-over-overlay';
        div.innerHTML = `<h3>mUø[æb ý </h3><p>• <ik!ep: ${this.errors}</p><button class="btn"
id="replayBtn">e°n8b </button>`;
        this.container.style.position = 'relative';
        this.container.appendChild(div);
        document.getElementById('replayBtn').addEventListener('click', () => {
            this.init(this.container, this.statsCb);
        });
    },

    destroy() {}
};

```

```
// === Flappy Bird (js/games/flappy.js) ===
```

```

/**
 * Flappy Bird Clone (\ ž ~p~b)
 * @author ch2077
 * @version 1.2.1
 * @date 2025-02-05 R rH
 * @date 2025-03-18 æ et'ÍR>SÄepÿ bKa fôc¥•ÑSÿrH
 *
 * ~İQxFlappy Birds@lõÿ p¹Qû<@ \ ž ~p•wz••Ç{i•S
 * rit ŷ kİ^'vy+=0.4'ÍR>ÿ p¹Qúvy=-6N Q²
 * {i•S•ô-"130pxÿ u b ~'s‡kİ80^'N [ù

```



```

*/
const FlappyGame = {
  canvas: null,
  ctx: null,
  width: 360,
  height: 500,
  bird: { x: 0, y: 0, r: 14, vy: 0 },
  pipes: [],
  score: 0,
  best: 0,
  running: false,
  started: false,
  loop: null,
  frameCount: 0,
  container: null,
  statsCb: null,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.score = 0;
    this.running = true;
    this.started = false;
    this.frameCount = 0;
    this.pipes = [];

    const w = Math.min(360, container.clientWidth - 48);
    const h = Math.min(500, w * 1.4);
    this.width = w;
    this.height = h;
    this.bird = { x: w * 0.25, y: h / 2, r: 14, vy: 0 };

    this.canvas = document.createElement('canvas');
    this.canvas.className = 'game-canvas';
    this.canvas.width = w;
    this.canvas.height = h;
    this.ctx = this.canvas.getContext('2d');

    this.container.innerHTML = '';
    this.container.appendChild(this.canvas);
    const inst = document.createElement('div');
    inst.className = 'instructions';
    inst.textContent = 'p'Qũ\0^U<@\\ ž ~p•wgeÿ ••_ {i•Sÿ ' ';
    this.container.appendChild(inst);

    this.canvas.addEventListener('click', () => this.tap());
    this.canvas.addEventListener('touchstart', (e) => { e.preventDefault(); this.tap(); });

    this.updateStats();
    this.draw();
    this.loop = setInterval(() => this.tick(), 16);
  },

  tap() {

```



```

    if (!this.running) return;
    if (!this.started) { this.started = true; }
    this.bird.vy = -6;
  },

  tick() {
    if (!this.running) return;
    this.frameCount++;

    if (this.started) {
      this.bird.vy += 0.4;
      this.bird.y += this.bird.vy;

      // Pipe generation
      if (this.frameCount % 80 === 0) {
        const gap = 130;
        const center = 60 + Math.random() * (this.height - gap - 120);
        this.pipes.push({ x: this.width, top: center - gap / 2, bottom: center + gap / 2, scored: false });
      }

      // Move pipes
      for (const p of this.pipes) p.x -= 2.5;

      // Remove off-screen pipes
      this.pipes = this.pipes.filter(p => p.x > -60);

      // Collision
      const b = this.bird;
      // Ground/ceiling
      if (b.y - b.r < 0 || b.y + b.r > this.height) { this.gameOver(); return; }

      for (const p of this.pipes) {
        if (b.x + b.r > p.x && b.x - b.r < p.x + 40) {
          if (b.y - b.r < p.top || b.y + b.r > p.bottom) { this.gameOver(); return; }
        }
        if (!p.scored && p.x + 20 < b.x) {
          p.scored = true;
          this.score++;
          this.updateStats();
        }
      }
    }

    this.draw();
  },

  draw() {
    const c = this.ctx;
    c.clearRect(0, 0, this.width, this.height);

    // Sky
    c.fillStyle = '#1a3a5c';
    c.fillRect(0, 0, this.width, this.height);
  }
};

```



```

// Pipes
for (const p of this.pipes) {
  c.fillStyle = '#00d2a0';
  c.fillRect(p.x, 0, 40, p.top);
  c.fillRect(p.x - 3, p.top - 25, 46, 25);
  c.fillRect(p.x, p.bottom, 40, this.height - p.bottom);
  c.fillRect(p.x - 3, p.bottom, 46, 25);
}

// Bird
c.fillStyle = '#ffd93d';
c.beginPath();
c.arc(this.bird.x, this.bird.y, this.bird.r, 0, Math.PI * 2);
c.fill();
c.fillStyle = '#fff';
c.beginPath();
c.arc(this.bird.x + 3, this.bird.y - 3, 5, 0, Math.PI * 2);
c.fill();
c.fillStyle = '#000';
c.beginPath();
c.arc(this.bird.x + 5, this.bird.y - 4, 2, 0, Math.PI * 2);
c.fill();

if (!this.started) {
  c.fillStyle = '#fff';
  c.font = '16px sans-serif';
  c.textAlign = 'center';
  c.fillText('p!Q0_ YË', this.width / 2, this.height / 2 - 40);
}
},

updateStats() {
  if (this.statsCb) {
    if (this.score > this.best) this.best = this.score;
    this.statsCb({ '_R': this.score, 'g 0s': this.best });
  }
},

gameOver() {
  this.running = false;
  clearInterval(this.loop);
  if (this.score > this.best) this.best = this.score;
  this.updateStats();
  this.draw();

  const div = document.createElement('div');
  div.className = 'game-over-overlay';
  div.innerHTML = `
    <h3>n8b ~Óg_</h3>
    <p>_R : ${this.score} | g 0s: ${this.best}</p>
    <button class="btn" id="replayBtn">Q•geN k!</button>
  `;
  this.container.style.position = 'relative';
  this.container.appendChild(div);

```



```

        document.getElementById('replayBtn').addEventListener('click', () => {
            this.destroy();
            this.init(this.container, this.statsCb);
        });
    },

    destroy() {
        this.running = false;
        clearInterval(this.loop);
    }
};

```

```
// === Fruit Catch (js/games/fruitcatch.js) ===
```

```

/**
 * Fruit Catch (c¥l4gœ)
 * @author ch2077
 * @version 1.1.2
 * @date 2025-02-28 R rH
 * @date 2025-03-22 R N†p, _9`é•Z+u T}|û~ß
 *
 * nÑR`{î[Pc¥00C%,=l4gœÿ ••_ p, _9
 * l4gœ8yíemoji-•g:ÿ p, _9i, s†15%ÿ • ^!-•eö•ô• Xž
 * 5gaT}ÿ o c¥l4gœ-1T}ÿ c¥R0p, _9vôc¥-1T}
 */
const FruitCatchGame = {
    canvas: null,
    ctx: null,
    width: 360,
    height: 550,
    basket: { x: 0, w: 80, h: 24 },
    items: [],
    score: 0,
    lives: 5,
    running: false,
    loop: null,
    frameCount: 0,
    speed: 1.5,
    container: null,
    statsCb: null,

    init(container, statsCb) {
        this.container = container;
        this.statsCb = statsCb;
        this.score = 0;
        this.lives = 5;
        this.running = true;
        this.frameCount = 0;
        this.items = [];
        this.speed = 1.5;

        const w = Math.min(360, container.clientWidth - 48);
        const h = Math.min(550, w * 1.5);
        this.width = w;

```



```

    this.height = h;
    this.basket = { x: w / 2 - 40, w: 80, h: 24 };

    this.canvas = document.createElement('canvas');
    this.canvas.className = 'game-canvas';
    this.canvas.width = w;
    this.canvas.height = h;
    this.ctx = this.canvas.getContext('2d');

    this.container.innerHTML = '';
    this.container.appendChild(this.canvas);
    const inst = document.createElement('div');
    inst.className = 'instructions';
    inst.textContent = ']æSónÑR`b p¹QûN$0$yûR`{î[Pc¥l4gø';
    this.container.appendChild(inst);

    // Touch/mouse controls
    this.canvas.addEventListener('touchmove', (e) => {
        e.preventDefault();
        const rect = this.canvas.getBoundingClientRect();
        const touchX = e.touches[0].clientX - rect.left;
        this.basket.x = touchX - this.basket.w / 2;
        this.clampBasket();
    });
    this.canvas.addEventListener('touchstart', (e) => {
        e.preventDefault();
        const rect = this.canvas.getBoundingClientRect();
        const touchX = e.touches[0].clientX - rect.left;
        this.basket.x = touchX - this.basket.w / 2;
        this.clampBasket();
    });
    this.canvas.addEventListener('mousemove', (e) => {
        const rect = this.canvas.getBoundingClientRect();
        this.basket.x = e.clientX - rect.left - this.basket.w / 2;
        this.clampBasket();
    });
    // Also allow clicking left/right halves
    this.canvas.addEventListener('click', (e) => {
        const rect = this.canvas.getBoundingClientRect();
        const clickX = e.clientX - rect.left;
        if (clickX < this.width / 2) {
            this.basket.x -= 40;
        } else {
            this.basket.x += 40;
        }
        this.clampBasket();
    });

    this.updateStats();
    this.loop = setInterval(() => this.tick(), 16);
},

clampBasket() {
    this.basket.x = Math.max(0, Math.min(this.width - this.basket.w, this.basket.x
));

```


$\},$

```
draw() {
  const c = this.ctx;
```



```

    c.clearRect(0, 0, this.width, this.height);
    c.fillStyle = '#1a1a3e';
    c.fillRect(0, 0, this.width, this.height);

    // Items
    for (const item of this.items) {
        c.font = '28px sans-serif';
        c.textAlign = 'center';
        c.fillText(item.emoji, item.x, item.y);
    }

    // Basket
    c.fillStyle = '#a29bfe';
    c.fillRect(this.basket.x, this.height - this.basket.h, this.basket.w, this.bas
ket.h);
    c.fillStyle = '#6c5ce7';
    c.fillRect(this.basket.x + 5, this.height - this.basket.h + 2, this.basket.w -
10, this.basket.h - 4);

    // Lives
    c.font = '14px sans-serif';
    c.textAlign = 'left';
    c.fillStyle = '#fff';
    c.fillText('dp '.repeat(this.lives), 8, 22);
},

updateStats() {
    if (this.statsCb) {
        this.statsCb({ '_R ': this.score, 'u T': this.lives });
    }
},

gameOver() {
    this.running = false;
    clearInterval(this.loop);

    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = `<h3>n8b ~0g_</h3><p>_R : ${this.score}</p><button class="btn" id="
replayBtn">Q•geN k!</button>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
        this.destroy();
        this.init(this.container, this.statsCb);
    });
},

destroy() {
    this.running = false;
    clearInterval(this.loop);
}
};

```



```
// === Sliding Puzzle (js/games/puzzle.js) ===
```

```
/**
 * Sliding Puzzle (15-Puzzle / nÑWbùVp)
 * @author ch2077
 * @version 1.1.0
 * @date 2025-02-15 R Qe• ^•epR$[šSi%ã`'
 * @date 2025-02-20 XžR shuffleeöV„Si%ãh!šŒÿ -2kbu b k{\@
 *
 * 4x4SAN"bùVpÿ ep[W1-15+zzOM
 * p¹QÛN zzOMvø•»v„nÑWwyûR`ÿ vÔR0c'R N:1-15~z^•
 * -•g:bSNqeöu(• ^•[üepR$e-Si%ã`'ÿ OÝ<ÁN O u b eà%ãvø-b
 */
const PuzzleGame = {
  size: 4,
  tiles: [],
  emptyIdx: 15,
  moves: 0,
  container: null,
  statsCb: null,
  gridEl: null,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.moves = 0;
    this.tiles = Array.from({ length: this.size * this.size - 1 }, (_, i) => i + 1
);
    this.tiles.push(0); // 0 = empty
    this.emptyIdx = this.tiles.indexOf(0);

    // Shuffle by making random valid moves
    for (let i = 0; i < 200; i++) {
      const neighbors = this.getNeighbors(this.emptyIdx);
      const pick = neighbors[Math.floor(Math.random() * neighbors.length)];
      this.swap(this.emptyIdx, pick);
      this.emptyIdx = pick;
    }

    this.render();
    this.updateStats();
  },

  getNeighbors(id) {
    const r = Math.floor(id / this.size);
    const c = id % this.size;
    const result = [];
    if (r > 0) result.push(id - this.size);
    if (r < this.size - 1) result.push(id + this.size);
    if (c > 0) result.push(id - 1);
    if (c < this.size - 1) result.push(id + 1);
    return result;
  },

  swap(a, b) {
```



```

    [this.tiles[a], this.tiles[b]] = [this.tiles[b], this.tiles[a]];
  },

  render() {
    this.container.innerHTML = '<div class="puzzle-grid"></div>';
    this.gridEl = this.container.querySelector('.puzzle-grid');

    this.tiles.forEach((val, i) => {
      const tile = document.createElement('div');
      tile.className = 'puzzle-tile' + (val === 0 ? ' empty' : '');
      tile.textContent = val || '';
      tile.addEventListener('click', () => this.move(i));
      this.gridEl.appendChild(tile);
    });
  },

  move(idx) {
    if (this.tiles[idx] === 0) return;
    const neighbors = this.getNeighbors(this.emptyIdx);
    if (!neighbors.includes(idx)) return;

    this.swap(idx, this.emptyIdx);
    this.emptyIdx = idx;
    this.moves++;
    this.updateStats();
    this.refreshDOM();

    if (this.checkWin()) {
      this.showWin();
    }
  },

  refreshDOM() {
    const tileEls = this.gridEl.querySelectorAll('.puzzle-tile');
    this.tiles.forEach((val, i) => {
      tileEls[i].textContent = val || '';
      tileEls[i].className = 'puzzle-tile' + (val === 0 ? ' empty' : '');
    });
  },

  checkWin() {
    for (let i = 0; i < this.tiles.length - 1; i++) {
      if (this.tiles[i] !== i + 1) return false;
    }
    return true;
  },

  updateStats() {
    if (this.statsCb) {
      this.statsCb({ 'keep': this.moves });
    }
  },

  showWin() {
    const div = document.createElement('div');

```



```

    div.className = 'game-over-overlay';
    div.innerHTML = `
      <h3>ð<ß% bűvp[Æb ý </h3>
      <p>u(N† ${this.moves} ke</p>
      <button class="btn" id="replayBtn">Q•geN \@</button>
    `;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
      this.init(this.container, this.statsCb);
    });
  },

  destroy() {}
};

// === Tic-Tac-Toe (js/games/tictactoe.js) ===

/**
 * Tic-Tac-Toe (N•[WhĚ)
 * @author ch2077
 * @version 1.0.3
 * @date 2025-01-25 R rH
 * @date 2025-02-18 Aİe9u(minimaxÿ NĪ-•g:„=[PSG~$N:N Sĭb €Ü
 *
 * 3x3N•[WhĚÿ s@[¶XQHbKvs AI(0)
 * AIO•u(minimax• _Rd }"g 0 %ăÿ 0Ÿ<Á•ó\ ^s\@
 * <°_U€Ü• ^s~ß<ıÿ wins/losses/drawsÿ
 */
const TicTacToeGame = {
  board: [],
  player: 'X',
  ai: 'O',
  turn: 'X',
  wins: 0,
  losses: 0,
  draws: 0,
  container: null,
  statsCb: null,
  gridEl: null,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.board = Array(9).fill('');
    this.turn = 'X';
    this.render();
    this.updateStats();
  },

  render() {
    this.container.innerHTML = `
      <div class="tictactoe-grid"></div>
      <div class="instructions" style="margin-top:16px">0`bg Xÿ AI bg 0</div>
    `;
  }
};
```



```

    `;
    this.gridEl = this.container.querySelector('.tictactoe-grid');
    this.gridEl.style.cssText = 'display:grid;grid-template-columns:repeat(3,80px)
;grid-template-rows:repeat(3,80px);gap:4px;';

    for (let i = 0; i < 9; i++) {
        const cell = document.createElement('div');
        cell.className = 'ttd-cell';
        cell.style.cssText = 'background:var(--surface2);border-radius:8px;display:flex;align-items:center;justify-content:center;font-size:36px;font-weight:700;cursor:pointer;transition:background 0.15s;';
        cell.dataset.idx = i;
        cell.textContent = this.board[i];
        cell.addEventListener('click', () => this.playerMove(i, cell));
        this.gridEl.appendChild(cell);
    }
},

playerMove(idx, cell) {
    if (this.board[idx] !== '' || this.turn !== 'X') return;
    this.board[idx] = 'X';
    cell.textContent = 'X';
    cell.style.color = '#54a0ff';

    if (this.checkWin('X')) {
        this.wins++;
        this.updateStats();
        this.endGame('O`•bN†ÿ ');
        return;
    }
    if (this.board.every(c => c !== '')) {
        this.draws++;
        this.updateStats();
        this.endGame('^s\@ÿ ');
        return;
    }

    this.turn = 'O';
    setTimeout(() => this.aiMove(), 300);
},

aiMove() {
    const move = this.getBestMove();
    this.board[move] = 'O';
    const cell = this.gridEl.querySelectorAll('.ttd-cell')[move];
    cell.textContent = 'O';
    cell.style.color = '#ff6b6b';

    if (this.checkWin('O')) {
        this.losses++;
        this.updateStats();
        this.endGame('AI •bN†ÿ ');
        return;
    }
    if (this.board.every(c => c !== '')) {

```



```

        this.draws++;
        this.updateStats();
        this.endGame('^s\@ÿ ');
        return;
    }
    this.turn = 'X';
},

getBestMove() {
    // Minimax
    let best = -Infinity;
    let move = 0;
    for (let i = 0; i < 9; i++) {
        if (this.board[i] === '') {
            this.board[i] = 'O';
            const score = this.minimax(0, false);
            this.board[i] = '';
            if (score > best) { best = score; move = i; }
        }
    }
    return move;
},

minimax(depth, isMax) {
    if (this.checkWin('O')) return 10 - depth;
    if (this.checkWin('X')) return depth - 10;
    if (this.board.every(c => c !== '')) return 0;

    if (isMax) {
        let best = -Infinity;
        for (let i = 0; i < 9; i++) {
            if (this.board[i] === '') {
                this.board[i] = 'O';
                best = Math.max(best, this.minimax(depth + 1, false));
                this.board[i] = '';
            }
        }
        return best;
    } else {
        let best = Infinity;
        for (let i = 0; i < 9; i++) {
            if (this.board[i] === '') {
                this.board[i] = 'X';
                best = Math.min(best, this.minimax(depth + 1, true));
                this.board[i] = '';
            }
        }
        return best;
    }
},

checkWin(p) {
    const wins = [[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],[0,4,8],[2,4,6]]
;
    return wins.some(w => w.every(i => this.board[i] === p));

```



```
    },

    updateStats() {
      if (this.statsCb) {
        this.statsCb({ '€': this.wins, '• ': this.losses, '^s': this.draws });
      }
    },

    endGame(msg) {
      this.gridEl.querySelectorAll('.ttt-cell').forEach(c => c.style.cursor = 'defau
lt');
      const div = document.createElement('div');
      div.className = 'game-over-overlay';
      div.innerHTML = `<h3>${msg}</h3><button class="btn" id="replayBtn">Q•geN \@\</butto
n>`;
      this.container.style.position = 'relative';
      this.container.appendChild(div);
      document.getElementById('replayBtn').addEventListener('click', () => {
        this.init(this.container, this.statsCb);
      });
    },

    destroy() {}
  };

// === Simon Says (js/games/simon.js) ===

/**
 * Simon Says (°_Æpo)
 * @author ch2077
 * @version 1.0.2
 * @date 2025-03-25 R rH
 * @date 2025-04-02 0 S •êpÁ•ô-”ÿ R ••^•R eökİke•ô-”• Qİ
 *
 * ~İQx<°_Æpon8b ÿ <°00poQI^•R ^vc ~z^•Y s°
 * kİ•n^•R ••^!+1ÿ g ••Sİeà-Pc b ÿ • <İSs-Ōg_
 * 4,rpoÿ ~¢„Ÿ~ŸŽÄÿ %Æ+ÿ 0*ÿ -óeHcBy:
 */
const SimonGame = {
  sequence: [], playerId: 0,
  showingSequence: false,
  score: 0, highScore: 0,
  container: null, statsCb: null,
  colors: ['#ff6b6b', '#54a0ff', '#00d2a0', '#ffd93d'],
  colorNames: ['~¢', '„Ÿ', '~ÿ', 'žÄ'],
  soundEnabled: true,
}

init(container, statsCb) {
  this.container = container;
  this.statsCb = statsCb;
  this.sequence = [];
  this.playerId = 0;
  this.showingSequence = false;
  this.score = 0;
}
```



```

    const saved = localStorage.getItem('simon_high');ð
    this.highScore = saved ? parseInt(saved) : 0;ð
    this.render();ð
    setTimeout(() => this.nextRound(), 600);ð
  },ð
ð
  render() {ð
    this.container.innerHTML = `ð
      <div style="text-align:center">ð
        <div style="color:var(--text2);font-size:13px;margin-bottom:12px"><°00poQI~z^•ÿ q6
T 'fY p¹Qûÿ </div>ð
        <div class="simon-board">ð
          ${this.colors.map((c, i) => `ð
            <button class="simon-btn" data-idx="${i}" style="background:${c};aspec
t-ratio:1;border:none;border-radius:16px;cursor:pointer;transition:all 0.15s;box-s
hadow:0 0 0 rgba(0,0,0,0.3)"></button>ð
            `).join('')}}ð
          </div>ð
          <div id="simonStatus" style="margin-top:14px;font-weight:600;font-size:15p
x;color:var(--accent2)">QÆY ...</div>ð
          <div style="color:var(--text2);font-size:12px;margin-top:8px">g š0R : ${this.
highScore}</div>ð
        </div>`;ð
  },ð
ð
  this.container.querySelectorAll('.simon-btn').forEach((btn, i) => {ð
    btn.addEventListener('click', () => this.playerPress(i, btn));ð
  });ð
},ð
ð
  nextRound() {ð
    this.showingSequence = true;ð
    this.playerIdx = 0;ð
    this.sequence.push(Math.floor(Math.random() * 4));ð
    document.getElementById('simonStatus').textContent = '<°00~z^•...';ð
    this.updateStats();ð
    this.playSequence(0);ð
  },ð
ð
  playSequence(i) {ð
    if (i >= this.sequence.length) {ð
      this.showingSequence = false;ð
      document.getElementById('simonStatus').textContent = '•nR00`ÿ ';ð
      return;ð
    }ð
    const idx = this.sequence[i];ð
    const btn = this.container.querySelector(`.simon-btn[data-idx="${idx}"]`);ð
    this.lightUp(btn, idx);ð
    setTimeout(() => this.playSequence(i + 1), 500);ð
  },ð
ð
  lightUp(btn, idx) {ð
    btn.style.transform = 'scale(1.1)';ð
    btn.style.boxShadow = `0 0 24px ${this.colors[idx]}`;ð
    // Simple tone via oscillatorð
    if (this.soundEnabled) {ð

```



```

    try {
      const ctx = new (window.AudioContext || window.webkitAudioContext)();
      const osc = ctx.createOscillator();
      const gain = ctx.createGain();
      osc.connect(gain); gain.connect(ctx.destination);
      osc.frequency.value = 300 + idx * 100;
      osc.type = 'sine';
      gain.gain.value = 0.1;
      osc.start(); osc.stop(ctx.currentTime + 0.15);
    } catch(e) {}
  }
  setTimeout(() => {
    btn.style.transform = 'scale(1)';
    btn.style.boxShadow = '0 0 0 rgba(0,0,0,0.3)';
  }, 300);
},
),
)
playerPress(idx, btn) {
  if (this.showingSequence) return;
  this.lightUp(btn, idx);
)
  if (idx !== this.sequence[this.playerIdx]) {
    document.getElementById('simonStatus').innerHTML = '<span style="color:#ff6b">• <i> </span>';
    if (this.score > this.highScore) {
      this.highScore = this.score;
      localStorage.setItem('simon_high', this.highScore);
    }
    setTimeout(() => this.showGameOver(), 800);
    return;
  }
)
  this.playerIdx++;
  if (this.playerIdx >= this.sequence.length) {
    this.score++;
    document.getElementById('simonStatus').textContent = 'kcxnÿ N N •n...';
    setTimeout(() => this.nextRound(), 800);
  }
  this.updateStats();
},
),
)
updateStats() {
  if (this.statsCb) this.statsCb({ '_R ': this.score, '^•R ••^|': this.sequence.length });
},
),
)
showGameOver() {
  const div = document.createElement('div');
  div.className = 'game-over-overlay';
  div.innerHTML = `<h3>n8b ~ôg_</h3><p>R ep: ${this.score} | g šØR : ${this.highScore}</p>
<button class="btn" id="replayBtn">Q•geN k!</button>`;
  this.container.style.position = 'relative';
  this.container.appendChild(div);
  document.getElementById('replayBtn').addEventListener('click', () => {
    this.init(this.container, this.statsCb);
  });
}

```



```

    });ð
  },ð
ð
  destroy() {}ð
};ð

```

```
// === Reaction (js/games/reaction.js) ===
```

```

/**
 * Reaction Time (SÍ^"• ^!mK<Ö)
 * @author ch2077
 * @version 1.0.3
 * @date 2025-02-01 R rH
 * @date 2025-03-08 R N†pre-clickhÀmKÿ -2kb~,R$çÐRMP¹Qû
 *
 * \0^US0~ÿT \=_ëp¹Qûÿ mK<ÖSÍ^"• ^!ÿ msÿ
 * r¶` g:ÿ idle!'waiting(random delay)!'ready!'clicked
 * <°_Ug •Ñ5k!b ~é+g 0sb ~é
 */
const ReactionGame = {
  state: 'idle', // idle, waiting, ready, clicked
  startTime: 0,
  timeout: null,
  best: Infinity,
  results: [],
  container: null,
  statsCb: null,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.state = 'idle';
    this.results = [];
    this.render();
    this.updateStats();
  },

  render() {
    this.container.innerHTML = `
      <div style="text-align:center">
        <div class="reaction-zone" id="reactionZone"
          style="width:200px;height:200px;border-radius:50%;margin:20px auto;displ
ay:flex;align-items:center;justify-content:center;cursor:pointer;transition:backgr
ound 0.1s;user-select:none;font-size:16px;font-weight:700">
          <span id="zoneText">p¹b _ YË</span>
        </div>
        <div id="reactionResults" style="margin-top:16px"></div>
      </div>
    `;

    const zone = document.getElementById('reactionZone');
    this.updateZoneStyle(zone, 'idle');

    zone.addEventListener('click', () => {

```



```

    if (this.state === 'idle' || this.state === 'clicked') {
      this.startCountdown();
    } else if (this.state === 'waiting') {
      this.tooEarly();
    } else if (this.state === 'ready') {
      this.recordReaction();
    }
  });
},

updateZoneStyle(zone, state) {
  if (state === 'idle') {
    zone.style.background = 'var(--accent)';
    zone.style.color = 'ffff';
    document.getElementById('zoneText').textContent = 'p¹b _ YĖ';
  } else if (state === 'waiting') {
    zone.style.background = '#ff6b6b';
    zone.style.color = 'ffff';
    document.getElementById('zoneText').textContent = '{I_...~ÿ,r...';
  } else if (state === 'ready') {
    zone.style.background = '#00d2a0';
    zone.style.color = 'ffff';
    document.getElementById('zoneText').textContent = '_ëc ÿ ';
  } else if (state === 'clicked') {
    zone.style.background = 'var(--accent)';
    zone.style.color = 'ffff';
    document.getElementById('zoneText').textContent = 'Q•<õN k!';
  }
},

startCountdown() {
  this.state = 'waiting';
  const zone = document.getElementById('reactionZone');
  this.updateZoneStyle(zone, 'waiting');

  const delay = 1000 + Math.random() * 3000;
  clearTimeout(this.timeout);
  this.timeout = setTimeout(() => {
    this.state = 'ready';
    this.startTime = Date.now();
    this.updateZoneStyle(zone, 'ready');
  }, delay);
},

tooEarly() {
  this.state = 'clicked';
  clearTimeout(this.timeout);
  const zone = document.getElementById('reactionZone');
  zone.style.background = '#ffa726';
  document.getElementById('zoneText').textContent = 'Y*eéN†ÿ {I~ÿ,rQ•c ';
  setTimeout(() => {
    this.state = 'idle';
    this.updateZoneStyle(zone, 'idle');
  }, 1500);
},

```



```

recordReaction() {
  this.state = 'clicked';
  const time = Date.now() - this.startTime;
  this.results.unshift(time);
  if (this.results.length > 5) this.results.pop();
  if (time < this.best) this.best = time;

  const zone = document.getElementById('reactionZone');
  document.getElementById('zoneText').textContent = `${time} ms`;
  this.updateStats();
  this.showResults();

  setTimeout(() => {
    this.state = 'idle';
    this.updateZoneStyle(zone, 'idle');
  }, 2000);
},

showResults() {
  const el = document.getElementById('reactionResults');
  el.innerHTML = `<p style="color:var(--text2);font-size:13px;margin-bottom:8px"
>g •Ñ 5 k!ÿ </p>
<div style="display:flex;gap:6px;justify-content:center;flex-wrap:wrap">
  ${this.results.map((t, i) => `<span style="padding:4px 10px;border-radius:
20px;font-size:12px;font-weight:600;
  background:${i===0?'var(--accent)':'var(--surface2)'};color:${i===0?'#ff
f':'var(--text)'}"
  >${t}ms</span>`).join('')}`
  </div>`;
},

updateStats() {
  if (this.statsCb) {
    this.statsCb({ 'g Os': this.best === Infinity ? '--' : `${this.best}ms` });
  }
},

destroy() {
  clearTimeout(this.timeout);
}
};

```

```
// === Number Guess (js/games/numberguess.js) ===
```

```
/**
```

```
 * Number Guessing (ep[Ws Ɛ )
```

```
 * @author ch2077
```

```
 * @version 1.0.2
```

```
 * @date 2025-01-28 R rH
```

```
 * @date 2025-02-08 f VôcBy:e9N:R" S:•ôf>y:
```

```
 *
```

```
 * s 1-100-•g:epÿ kİk!s [ƐcĐy:"Y'N†/\ N†"^v•)\ f Vô
```

```
 * <jk!~ß<ÿ cĐO>'İ•n'İe°_ \@
```



```

*/
const NumberGuessGame = {
  target: 0,
  attempts: 0,
  range: [1, 100],
  container: null,
  statsCb: null,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.target = Math.floor(Math.random() * 100) + 1;
    this.attempts = 0;
    this.range = [1, 100];
    this.render();
    this.updateStats();
  },

  render() {
    this.container.innerHTML = `
      <div style="text-align:center">
        <div style="font-size:60px;margin-bottom:12px">0>Ÿ </div>
        <p style="color:var(--text2);margin-bottom:8px">b `0N†N N* 1~100 NK•ôv„ep[W</p>
        <p style="color:var(--accent2);font-size:14px;margin-bottom:20px" id="rang
eHint">f Vô: 1 ~ 100</p>
        <div style="display:flex;gap:8px;justify-content:center;margin-bottom:16px
">
          <input type="number" id="guessInput" min="1" max="100" placeholder="•“Qeep[W
"
          style="width:140px;padding:10px 14px;border-radius:8px;border:1px soli
d var(--border);background:var(--surface2);color:var(--text);font-size:16px;text-a
lign:center">
          <button class="btn" id="guessBtn">s ŷ </button>
        </div>
        <div id="resultHint" style="font-size:15px;min-height:24px"></div>
        <div id="history" style="margin-top:12px;display:flex;flex-wrap:wrap;gap:6
px;justify-content:center"></div>
      </div>
    `;

    const input = document.getElementById('guessInput');
    document.getElementById('guessBtn').addEventListener('click', () => this.guess
());
    input.addEventListener('keydown', (e) => {
      if (e.key === 'Enter') this.guess();
    });
    input.focus();
  },

  guess() {
    const input = document.getElementById('guessInput');
    const val = parseInt(input.value);
    if (isNaN(val) || val < 1 || val > 100) {
      document.getElementById('resultHint').innerHTML = '<span style="color:#ff6b6
b">÷•“Qe 1~100 v„ep[W</span>';

```



```

    return;
  }

  this.attempts++;
  this.updateStats();

  const hint = document.getElementById('resultHint');
  const history = document.getElementById('history');
  const badge = document.createElement('span');
  badge.style.cssText = 'padding:4px 10px;border-radius:20px;font-size:12px;font
-weight:600;';

  if (val === this.target) {
    hint.innerHTML = `<span style="color:#00d2a0;font-size:18px">ð<ð% s [ùN†ÿ ep[W\1f/ ${t
his.target}</span>`;
    badge.style.background = 'var(--green)';
    badge.style.color = '#fff';
    badge.textContent = val + ' ' ' ';
    history.appendChild(badge);
    this.showWin();
    return;
  }

  if (val < this.target) {
    this.range[0] = Math.max(this.range[0], val + 1);
    hint.innerHTML = `<span style="color:#ffa726">ð=ÜÉ Y*\\ N†ÿ Q•Y'N p¹</span>`;
    badge.style.background = '#3a2a10';
    badge.style.color = '#ffa726';
  } else {
    this.range[1] = Math.min(this.range[1], val - 1);
    hint.innerHTML = `<span style="color:#54a0ff">ð=ÜÉ Y*Y'N†ÿ Q•\\ N p¹</span>`;
    badge.style.background = '#1a2a3a';
    badge.style.color = '#54a0ff';
  }
  badge.textContent = val;
  history.appendChild(badge);

  document.getElementById('rangeHint').textContent = `f Vð: ${this.range[0]} ~ ${t
his.range[1]}`;
  input.value = '';
  input.focus();
},

updateStats() {
  if (this.statsCb) {
    this.statsCb({ 's mkk!ep': this.attempts });
  }
},

showWin() {
  document.getElementById('guessBtn').disabled = true;
  document.getElementById('guessInput').disabled = true;
  const div = document.createElement('div');
  div.className = 'game-over-overlay';
  div.innerHTML = `<h3>ð<ð% s [ùN†ÿ </h3><p>u(N† ${this.attempts} k!</p><button class="bt

```



```

        this.container.style.position = 'relative';
        this.container.appendChild(div);
        document.getElementById('replayBtn').addEventListener('click', () => {
            this.init(this.container, this.statsCb);
        });
    },

    destroy() {}
};

// === Color Match (js/games/colormatch.js) ===

/**
 * Color Match / Stroop Test (~æ,rS9'M)
 * @author ch2077
 * @version 1.0.4
 * @date 2025-02-12 R rHÿ WÚNŽStroopeH^"ÿ
 * @date 2025-03-10 30y0-Peö+•PQÛR b
 *
 * R$e-e‡[W~æ,rƒ/T&N e‡[WQ…[¹N •öÿ 30y0-Peö
 * StroopeH^"ÿ e‡[W<ö"-¢"Ofƒ>y:„Ÿ,r!'SÍ^"S0ab
 * •PQÛ(Streak)<¡k!ÿ g š0•PQÛ<°_U0Ÿ[X
 */
const ColorMatchGame = {
    colorName: '',
    displayColor: '',
    score: 0,
    streak: 0,
    bestStreak: 0,
    timeLeft: 30,
    running: false,
    timer: null,
    container: null,
    statsCb: null,
    colors: ['~¢','„Ÿ','-ÿ','žÄ','jY','}+'],
    colorMap: { '~¢': '#ff6b6b', '„Ÿ': '#54a0ff', '-ÿ': '#00d2a0', 'žÄ': '#ffd93d', 'jY': '#ffa726', '}' + ': '#a29bfe' },

    init(container, statsCb) {
        this.container = container;
        this.statsCb = statsCb;
        this.score = 0;
        this.streak = 0;
        this.bestStreak = 0;
        this.timeLeft = 30;
        this.running = true;
        this.render();
        this.nextRound();
        this.startTimer();
        this.updateStats();
    },

    render() {
        this.container.innerHTML = `

```



```

    <div style="text-align:center">
      <div style="color:var(--text2);font-size:14px;margin-bottom:8px">e‡[W~æ,rN T yðf/T&
N •ðÿ </div>
      <div id="colorDisplay" style="font-size:48px;font-weight:800;min-height:80
px;display:flex;align-items:center;justify-content:center;transition:all 0.15s"></
div>
      <div style="display:flex;gap:12px;justify-content:center;margin-top:20px">
        <button class="btn" id="matchYes" style="font-size:18px;padding:14px 36p
x;background:var(--green)">' N •ð</button>
        <button class="btn" id="matchNo" style="font-size:18px;padding:14px 36px
;background:var(--red)">'L N N •ð</button>
      </div>
      <div id="matchResult" style="margin-top:16px;min-height:28px;font-size:15p
x;font-weight:600"></div>
      <div style="margin-top:12px;color:var(--text2);font-size:13px">
        •þQÛ: <span id="streakDisplay" style="color:var(--accent2);font-weight:700
">0</span>
      </div>
    </div>
  `;

  document.getElementById('matchYes').addEventListener('click', () => this.ansew
r(true));
  document.getElementById('matchNo').addEventListener('click', () => this.answer
(false));
},

nextRound() {
  if (!this.running) return;

  // Randomly choose whether color matches name
  const match = Math.random() < 0.4; // 40% chance of match
  this.colorName = this.colors[Math.floor(Math.random() * this.colors.length)];

  if (match) {
    this.displayColor = this.colorMap[this.colorName];
  } else {
    // Pick a different color
    const others = this.colors.filter(c => c !== this.colorName);
    const pick = others[Math.floor(Math.random() * others.length)];
    this.displayColor = this.colorMap[pick];
  }

  const display = document.getElementById('colorDisplay');
  if (display) {
    display.textContent = this.colorName;
    display.style.color = this.displayColor;
  }
},

answer(userSaysMatch) {
  if (!this.running) return;
  const actualMatch = this.displayColor === this.colorMap[this.colorName];
  const correct = (userSaysMatch === actualMatch);

```



```

    const resultEl = document.getElementById('matchResult');
    if (correct) {
      this.score += 10 + this.streak * 2;
      this.streak++;
      if (this.streak > this.bestStreak) this.bestStreak = this.streak;
      resultEl.innerHTML = '<span style="color:#00d2a0">kcxnÿ </span>';
    } else {
      this.streak = 0;
      resultEl.innerHTML = '<span style="color:#ff6b6b">• N†ÿ </span>';
    }

    document.getElementById('streakDisplay').textContent = this.streak;
    this.updateStats();
    this.nextRound();
  },

  startTimer() {
    this.timer = setInterval(() => {
      this.timeLeft--;
      this.updateStats();
      if (this.timeLeft <= 0) {
        this.gameOver();
      }
    }, 1000);
  },

  updateStats() {
    if (this.statsCb) {
      this.statsCb({ '_-R ': this.score, '•bQû': this.streak, 'eö•ô': `${this.timeLeft}s`
    });
    }
  },

  gameOver() {
    this.running = false;
    clearInterval(this.timer);
    if (this.streak > this.bestStreak) this.bestStreak = this.streak;

    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = `
      <h3>eö•ôR0ÿ </h3>
      <p>_-R : ${this.score} | g 0s•bQû: ${this.bestStreak}</p>
      <button class="btn" id="replayBtn">Q•geN k!</button>
    `;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
      this.destroy();
      this.init(this.container, this.statsCb);
    });
  },

  destroy() {
    this.running = false;

```



```
        clearInterval(this.timer);
    }
};

// === Stylesheet (css/style.css) ===

/* === Reset & Base === */
*, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
:root {
    --bg: #0f0f1a;
    --surface: #1a1a2e;
    --surface2: #252542;
    --border: #333355;
    --text: #e0e0f0;
    --text2: #8888aa;
    --accent: #6c5ce7;
    --accent2: #a29bfe;
    --green: #00d2a0;
    --red: #ff6b6b;
    --orange: #ffa726;
    --yellow: #ffd93d;
    --blue: #54a0ff;
    --radius: 12px;
    --shadow: 0 4px 24px rgba(0,0,0,0.3);
    --transition: 0.2s ease;
}
html, body {
    height: 100%;
    font-family: 'Segoe UI', system-ui, -apple-system, sans-serif;
    background: var(--bg);
    color: var(--text);
    overflow-x: hidden;
}
#app {
    min-height: 100vh;
    display: flex;
    flex-direction: column;
}
/* === Header === */
.app-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 16px 24px;
    background: var(--surface);
    border-bottom: 1px solid var(--border);
    position: sticky;
    top: 0;
    z-index: 100;
    backdrop-filter: blur(10px);
}
```



```
}@
@
.header-left { display: flex; align-items: center; gap: 12px; }@
.logo { font-size: 28px; }@
.app-header h1 { font-size: 20px; font-weight: 700; color: var(--text); }@
@
.btn-back {@
  background: var(--surface2);@
  color: var(--text);@
  border: 1px solid var(--border);@
  padding: 8px 16px;@
  border-radius: 8px;@
  cursor: pointer;@
  font-size: 14px;@
  transition: var(--transition);@
}@
.btn-back:hover { background: var(--border); }@
@
/* === Home Page === */@
.home-page { padding: 40px 24px; max-width: 1000px; margin: 0 auto; width: 100%; }
@
.hero {@
  text-align: center;@
  margin-bottom: 40px;@
}@
.hero h2 { font-size: 28px; margin-bottom: 8px; color: var(--accent2); }@
.hero p { font-size: 16px; color: var(--text2); }@
@
.game-grid {@
  display: grid;@
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));@
  gap: 20px;@
}@
@
.game-card {@
  background: var(--surface);@
  border: 1px solid var(--border);@
  border-radius: var(--radius);@
  padding: 24px;@
  cursor: pointer;@
  transition: transform var(--transition), box-shadow var(--transition), border-co
lor var(--transition);@
  display: flex;@
  flex-direction: column;@
  gap: 12px;@
}@
.game-card:hover {@
  transform: translateY(-4px);@
  box-shadow: var(--shadow);@
  border-color: var(--accent);@
}@
@
.game-card .game-icon { font-size: 40px; }@
.game-card .game-name { font-size: 18px; font-weight: 600; }@
.game-card .game-desc { font-size: 13px; color: var(--text2); line-height: 1.5; }@
```



```
.game-card .game-tags { display: flex; gap: 6px; flex-wrap: wrap; }  
.  
.game-card .tag {  
  font-size: 11px;  
  padding: 3px 10px;  
  border-radius: 20px;  
  background: var(--surface2);  
  color: var(--text2);  
  border: 1px solid var(--border);  
}  
  
/* === Game Page === */  
.game-page { padding: 20px 24px 40px; max-width: 700px; margin: 0 auto; width: 100%; }  
  
.game-header {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  flex-wrap: wrap;  
  gap: 12px;  
  margin-bottom: 20px;  
}  
  
.game-header h2 { font-size: 22px; color: var(--accent2); }  
  
.game-controls { display: flex; align-items: center; gap: 12px; flex-wrap: wrap; }  
  
.game-stats { display: flex; gap: 16px; }  
  
.stat {  
  background: var(--surface2);  
  padding: 6px 14px;  
  border-radius: 8px;  
  font-size: 13px;  
  color: var(--text2);  
}  
  
.stat span { color: var(--text); font-weight: 600; }  
  
.btn {  
  background: var(--accent);  
  color: #fff;  
  border: none;  
  padding: 8px 18px;  
  border-radius: 8px;  
  cursor: pointer;  
  font-size: 14px;  
  font-weight: 600;  
  transition: var(--transition);  
}  
  
.btn:hover { opacity: 0.85; transform: scale(1.02); }  
.btn:active { transform: scale(0.98); }  
.btn.small { padding: 5px 12px; font-size: 12px; }  
  
.game-area {  
  background: var(--surface);
```



```
border: 1px solid var(--border);␣
border-radius: var(--radius);␣
padding: 24px;␣
min-height: 400px;␣
display: flex;␣
flex-direction: column;␣
align-items: center;␣
justify-content: center;␣
}␣
␣
/* === Footer === */␣
.app-footer {␣
  text-align: center;␣
  padding: 20px;␣
  color: var(--text2);␣
  font-size: 13px;␣
  border-top: 1px solid var(--border);␣
  margin-top: auto;␣
}␣
␣
/* === Memory Game === */␣
.memory-grid {␣
  display: grid;␣
  gap: 8px;␣
  justify-content: center;␣
  justify-items: stretch;␣
}␣
␣
.memory-card {␣
  width: 72px;␣
  height: 72px;␣
  background: var(--surface2);␣
  border: 2px solid var(--border);␣
  border-radius: 10px;␣
  cursor: pointer;␣
  font-size: 32px;␣
  display: flex;␣
  align-items: center;␣
  justify-content: center;␣
  transition: transform 0.3s ease, background var(--transition);␣
  user-select: none;␣
}␣
.memory-card:hover { border-color: var(--accent); }␣
.memory-card.flipped { background: var(--accent); border-color: var(--accent2); }␣
.memory-card.matched { background: var(--green); border-color: var(--green); opacity: 0.7; }␣
␣
/* === 2048 === */␣
.game-2048 { width: 100%; max-width: 400px; }␣
.board-2048 {␣
  display: grid;␣
  grid-template-columns: repeat(4, 1fr);␣
  grid-template-rows: repeat(4, 1fr);␣
  gap: 8px;␣
  aspect-ratio: 1;␣
```



```
background: var(--surface2));␣
padding: 8px;␣
border-radius: var(--radius);␣
}␣
␣
.cell-2048 {␣
  background: var(--bg);␣
  border-radius: 8px;␣
  display: flex;␣
  align-items: center;␣
  justify-content: center;␣
  font-size: 22px;␣
  font-weight: 700;␣
  transition: all 0.1s ease;␣
}␣
␣
/* === Minesweeper === */␣
.mine-grid {␣
  display: grid;␣
  gap: 2px;␣
  background: var(--surface2));␣
  padding: 4px;␣
  border-radius: var(--radius);␣
}␣
␣
.mine-cell {␣
  width: 32px;␣
  height: 32px;␣
  background: var(--bg);␣
  border: 1px solid var(--border);␣
  border-radius: 4px;␣
  display: flex;␣
  align-items: center;␣
  justify-content: center;␣
  font-size: 13px;␣
  font-weight: 700;␣
  cursor: pointer;␣
  user-select: none;␣
  transition: background 0.1s;␣
}␣
.mine-cell:hover { background: #2a2a4a; }␣
.mine-cell.revealed { background: var(--surface2); }␣
.mine-cell.flagged { background: var(--orange); }␣
.mine-cell.mine { background: var(--red); }␣
␣
/* === Sliding Puzzle === */␣
.puzzle-grid {␣
  display: grid;␣
  grid-template-columns: repeat(4, 70px);␣
  grid-template-rows: repeat(4, 70px);␣
  gap: 4px;␣
  background: var(--surface2));␣
  padding: 8px;␣
  border-radius: var(--radius);␣
}␣
```



```

    ␣
    .puzzle-tile {␣
      background: var(--accent);␣
      color: #fff;␣
      border-radius: 8px;␣
      display: flex;␣
      align-items: center;␣
      justify-content: center;␣
      font-size: 20px;␣
      font-weight: 700;␣
      cursor: pointer;␣
      transition: transform 0.15s ease, background var(--transition);␣
      user-select: none;␣
    }␣
    .puzzle-tile:hover { opacity: 0.85; }␣
    .puzzle-tile.empty { background: transparent; cursor: default; }␣
    ␣
    /* === Snake / Tetris Canvas === */␣
    canvas.game-canvas {␣
      border-radius: 8px;␣
      display: block;␣
      background: var(--bg);␣
    }␣
    ␣
    .instructions {␣
      text-align: center;␣
      color: var(--text2);␣
      font-size: 13px;␣
      margin-top: 12px;␣
    }␣
    ␣
    /* === Game Over Overlay === */␣
    .game-over-overlay {␣
      position: absolute;␣
      inset: 0;␣
      background: rgba(0,0,0,0.7);␣
      display: flex;␣
      flex-direction: column;␣
      align-items: center;␣
      justify-content: center;␣
      border-radius: var(--radius);␣
      gap: 16px;␣
    }␣
    .game-over-overlay h3 { font-size: 24px; }␣
    ␣
    /* === Level Transition Overlay === */␣
    .level-intro {␣
      position: absolute;␣
      inset: 0;␣
      display: flex;␣
      flex-direction: column;␣
      align-items: center;␣
      justify-content: center;␣
      background: rgba(0,0,0,0.82);␣
      border-radius: var(--radius);␣

```



```

    z-index: 20;
    animation: levelFade 1.5s ease forwards;
    pointer-events: none;
}
.level-intro .level-num {
    font-size: 42px;
    font-weight: 700;
    color: var(--accent2);
    text-shadow: 0 0 20px rgba(108,92,231,0.5);
}
.level-intro .level-title {
    font-size: 16px;
    color: var(--text);
    margin-top: 8px;
    opacity: 0.8;
}
@keyframes levelFade {
    0% { opacity: 1; }
    65% { opacity: 1; }
    100% { opacity: 0; }
}
/* === Responsive === */
@media (max-width: 600px) {
    .app-header { padding: 12px 16px; }
    .app-header h1 { font-size: 17px; }
    .hero h2 { font-size: 22px; }
    .home-page { padding: 24px 16px; }
    .game-page { padding: 16px; }
    .game-grid { grid-template-columns: 1fr; }
    .game-area { padding: 16px; }

    .memory-card { width: 58px; height: 58px; font-size: 24px; }

    .puzzle-grid {
        grid-template-columns: repeat(4, 60px);
        grid-template-rows: repeat(4, 60px);
    }
    .puzzle-tile { font-size: 18px; }

    .mine-cell { width: 28px; height: 28px; font-size: 11px; }

    .cell-2048 { font-size: 18px; }

    /* === Tile colors for 2048 === */
    .tile-2 { background: #eee4da; color: #776e65; }
    .tile-4 { background: #ede0c8; color: #776e65; }
    .tile-8 { background: #f2b179; color: #f9f6f2; }
    .tile-16 { background: #f59563; color: #f9f6f2; }
    .tile-32 { background: #f67c5f; color: #f9f6f2; }
    .tile-64 { background: #f65e3b; color: #f9f6f2; }
    .tile-128 { background: #edcf72; color: #f9f6f2; }
    .tile-256 { background: #edcc61; color: #f9f6f2; }

```



```
.tile-512 { background: #edc850; color: #f9f6f2; }  
.tile-1024 { background: #edc53f; color: #f9f6f2; }  
.tile-2048 { background: #edc22e; color: #f9f6f2; }  
  
/* Position relative for game area to support overlays */  
.game-area { position: relative; }  
  
/* === Delta Force Randomizer === */  
.delta-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(140px, 1fr));  
  gap: 12px;  
  width: 100%;  
}  
  
.delta-slot {  
  background: var(--surface2);  
  border: 2px solid var(--border);  
  border-radius: var(--radius);  
  padding: 12px;  
  text-align: center;  
  transition: border-color var(--transition), box-shadow var(--transition);  
}  
  
.delta-photo {  
  width: 100%;  
  aspect-ratio: 4/3;  
  border-radius: 8px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  transition: background 0.15s ease, transform 0.15s ease;  
  margin-bottom: 8px;  
  background: var(--bg);  
}  
  
.delta-photo-icon {  
  font-size: 36px;  
  filter: drop-shadow(0 2px 4px rgba(0,0,0,0.4));  
}  
  
.photo-done {  
  transform: scale(1.05);  
  box-shadow: 0 0 16px rgba(108,92,231,0.4);  
}  
  
.delta-label-sm {  
  font-size: 12px;  
  color: var(--text2);  
  margin-bottom: 4px;  
}  
  
.delta-display {  
  font-size: 14px;  
  font-weight: 700;
```



```
    min-height: 20px;@
    transition: color 0.1s;@
}@
@
.delta-display.rolling {@
  animation: deltaShake 0.06s infinite;@
}@
@
.delta-display.done {@
  animation: deltaPop 0.3s ease;@
}@
@
@keyframes deltaShake {@
  0% { transform: translateY(0); }@
  50% { transform: translateY(-2px); }@
  100% { transform: translateY(0); }@
}@
@
@keyframes deltaPop {@
  0% { transform: scale(1); }@
  50% { transform: scale(1.3); }@
  100% { transform: scale(1); }@
}@
@
@keyframes pulse {@
  0%, 100% { box-shadow: 0 0 0 0 rgba(108,92,231,0.4); }@
  50% { box-shadow: 0 0 0 12px rgba(108,92,231,0); }@
}@
@
.btn.pulse {@
  animation: pulse 2s infinite;@
}@
@
.delta-result-card {@
  background: var(--surface2);@
  padding: 16px;@
  border-radius: 12px;@
  text-align: left;@
  line-height: 2;@
  border: 1px solid var(--border);@
}@
@
.delta-result-item {@
  display: flex;@
  align-items: center;@
  gap: 8px;@
  padding: 4px 0;@
  border-bottom: 1px solid var(--border);@
}@
.delta-result-item:last-child { border-bottom: none; }@
@
.delta-result-icon { font-size: 18px; }@
.delta-result-name { color: var(--text2); font-size: 13px; min-width: 40px; }@
.delta-result-val { color: var(--accent2); font-weight: 700; margin-left: auto; }@
@
```



```
@media (max-width: 600px) {  
  .delta-grid {  
    grid-template-columns: repeat(2, 1fr);  
    gap: 8px;  
  }  
  .delta-photo-icon { font-size: 28px; }  
  .delta-display { font-size: 12px; }  
}  
  
/* === Wordle === */  
.wordle-grid { display: flex; flex-direction: column; gap: 6px; align-items: center; }  
.wordle-row { display: flex; gap: 6px; }  
.wordle-cell {  
  width: 52px; height: 52px;  
  border: 2px solid var(--border);  
  border-radius: 6px;  
  display: flex; align-items: center; justify-content: center;  
  font-size: 22px; font-weight: 700;  
  transition: background 0.3s, transform 0.4s;  
  text-transform: uppercase;  
}  
.wordle-key {  
  flex: 1; min-width: 24px; padding: 8px 4px !important;  
  font-size: 11px !important; font-weight: 600;  
  background: var(--surface2) !important;  
  color: var(--text) !important;  
  border: 1px solid var(--border);  
}  
  
/* === Simon Says === */  
.simon-board {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 12px;  
  max-width: 260px;  
  margin: 0 auto;  
}  
  
/* === Whack-a-Mole === */  
.whack-grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 10px;  
  max-width: 330px;  
  margin: 0 auto;  
}  
.whack-hole {  
  position: relative;  
  aspect-ratio: 1;  
  background: var(--surface2);  
  border-radius: 50%;  
  cursor: pointer;  
  overflow: hidden;  
  display: flex;
```



```
        align-items: center;
        justify-content: center;
    }
    .whack-mole {
        font-size: 36px;
        transform: translateY(60px);
        transition: transform 0.15s ease;
        z-index: 2;
    }
    .whack-mole.up { transform: translateY(0); }
    .whack-dirt {
        position: absolute; bottom: 0; left: 0; right: 0; height: 40%;
        background: #3d2e1a;
        border-radius: 50% 50% 0 0;
    }
    @media (max-width: 600px) {
        .wordle-cell { width: 40px; height: 40px; font-size: 18px; }
    }
    /* === D-Pad (Snake) === */
    .dpad {
        display: grid;
        grid-template-areas: ". up ." "left . right" ". down .";
        grid-template-columns: repeat(3, 56px);
        grid-template-rows: repeat(3, 56px);
        gap: 4px;
        justify-content: center;
        margin-top: 12px;
    }
    .dpad-btn {
        background: var(--surface2);
        color: var(--text);
        border: 2px solid var(--border);
        border-radius: 10px;
        font-size: 22px;
        cursor: pointer;
        user-select: none;
        -webkit-user-select: none;
        touch-action: manipulation;
    }
    .dpad-btn.active { background: var(--accent); }
    .dpad-up { grid-area: up; }
    .dpad-down { grid-area: down; }
    .dpad-left { grid-area: left; }
    .dpad-right { grid-area: right; }
    /* === Tetris Controls === */
    .tetris-controls {
        display: flex;
        gap: 8px;
        justify-content: center;
        margin-top: 12px;
        flex-wrap: wrap;
    }
```



```
.tet-btn {
  flex: 1;
  min-width: 48px;
  padding: 12px 8px !important;
  font-size: 18px !important;
  touch-action: manipulation;
  user-select: none;
  -webkit-user-select: none;
}
/* Hide D-pad & tetris controls on desktop, show on touch */
@media (hover: hover) and (pointer: fine) {
  .dpad { display: none; }
  .tetris-controls { display: none; }
}
```

